

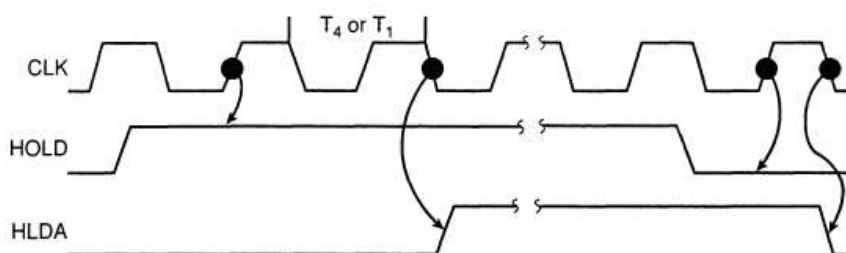
8. Структура на система с DMA контролер. Организация на обмена. Приложение. Стандартни интерфейси в PC и ARM кита. 64-битови архитектури. Тенденции за развитие на микропроцесорите.

Техниката на DMA /Директен достъп до паметта/ I/O осигурява пряк достъп до паметта, докато микропроцесорът е временно неактивен. Това позволява прехвърлянето на данни между паметта и I/O устройството със скорост, която е ограничена само от скоростта на компонентите на паметта в системата или DMA контролера. Скоростта на трансфер на DMA може да достигне скорост на трансфер от 15-20 М-байта/с при днешните високоскоростни компоненти на RAM паметта. DMA трансферите се използват за много цели, но по-често срещани са опресняването на DRAM, видеодисплеите за опресняване на екрана и в системата за четене и запис на дисковата памет. DMA трансферът се използва също за извършване на високоскоростни трансфери памет-памет.

Основни операции на DMA

Два управляващи сигнала, които са изведени на крачета на цокъла, се използват за искане и потвърждаване на прехвърляне с директен достъп до паметта (DMA) в микропроцесорна система. Сигналят HOLD е вход, използван за заявка на DMA, а HLDA е изходен сигнал, който потвърждава DMA действието. Фигура 12-1 показва типичната времедиаграма на тези два DMA контролни извода. Всеки път, когато на входа HOLD/задържане/ се получи ниво логическа 1, се изисква започване на DMA трансфер.

FIGURE 12-1 HOLD and HLDA timing for the 8086/8088 microprocessors



Микропроцесорът отговаря в рамките на няколко такта, като спира изпълнението на програмата и като постави своите адресни, даннови и контролни шини в техните високоимпедансни състояния. Това състояние изглежда така, сякаш кара микропроцесорът е изваден от гнездото си. То позволява на външни I/O устройства или други микропроцесори да получат достъп до системните шини, така че паметта може да бъде достъпена директно. Както показва диаграмата на времето, сигналят HOLD се проверява в средата на всеки тактов цикъл. По този начин задържането може да влезе в сила по всяко време по време на операцията на която и да е инструкция в микропроцесора. Веднага след като микропроцесорът разпознае активен сигнал HOLD, той спира да изпълнява софтуера и влиза в цикли на задържане. Входът HOLD има по-висок приоритет от входовете за прекъсване INTR или NMI. Прекъсванията влизат в сила в края на инструкцията, докато HOLD влиза в сила в средата на инструкцията. Единственото краче на микропроцесора, което има по-висок приоритет от HOLD, е краче RESET. Входът HOLD трябва да не е активен по време на RESET, иначе НУ не е гарантирано.

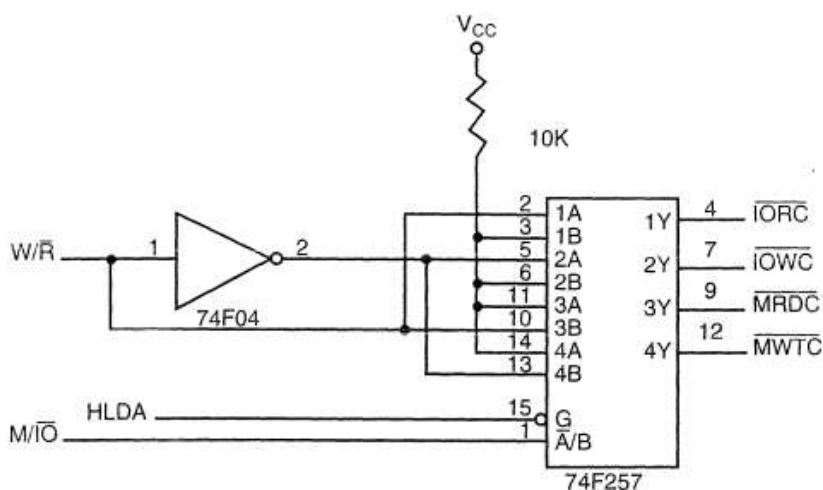
Сигналят HLDA става активен, за да покаже, че микропроцесорът наистина е поставил своите шини в техните състояния с висок импеданс, както може да се види на времевата диаграма. Има няколко тактови цикъла между времето, в което се променя HOLD, до момента, в който се променя HLDA. HLDA изходът е сигнал към външното искащо устройство, че микропроцесорът се е отказал от контрола върху своята памет и

I/O пространство. Можете да наречете входа HOLD вход за DMA заявка и HLDA изхода DMA сигнал за предоставяне на шините.

Основни DMA дефиниции

Директният достъп до паметта обикновено се осъществява между I/O устройство и паметта без използването на микропроцесора. DMA четенето прехвърля данни от паметта към I/O устройството. DMA записът прехвърля данни от I/O устройство към паметта. И при двете операции паметта и I/O устройство се управляват едновременно, поради което системата съдържа отделни сигнали за управление на паметта и I/O. Тази специална структура на контролната шина на микропроцесора позволява DMA трансфера. Наличието на DMA четене води до активиране на сигналите MRDC и IOWC, прехвърляйки данни от паметта към I/O устройството. DMA записът води до активиране на сигналите MWTC и IORC. Тези сигнали от контролната шина са достъпни за всички микропроцесори от семейството на Intel, с изключение на системата 8086/8088. 8086/8088 изисква тяхното генериране или със системен контролер, или с верига като тази, илюстрирана на Фигура 12-2. DMA контролерът генерира адреса на паметта, а сигналът от контролера (DACK) избира I/O устройството по време на DMA трансфера. Скоростта на трансфер на данни се определя от скоростта на устройството с памет или DMA контролера, който често управлява DMA трансферите. Ако скоростта на паметта/времето за достъп/ е 100 ns, DMA трансферите се извършват със скорост до 1/100 ns или 10 М-байта в секунда. Ако DMA контролерът в системата функционира при максимална скорост от 5 MHz и ние все още използваме памет от 100 ns, максималната скорост на трансфер е 5 MHz, защото (DMA контролерът е по-бавен от паметта. В много случаи DMA контролерът забавя скоростта на системата, когато се извършват DMA трансфери.

FIGURE 12-2 A circuit that generates system control signals in a DMA environment



DMA контролер I8237

DMA контролерът 8237 изработва за паметта I/O сигнали за управление и адрес на паметта по време на DMA трансфера. 8237 всъщност е микропроцесор със специална цел, да осигури цялата работа по високоскоростен трансфер на данни между паметта и I/O. Фигура 12-3 показва изводите и блоковата диаграма на програмируемия DMA контролер 8237. Това устройство може да не се появи като дискретен компонент в съвременните микропроцесорни системи, то се появява в чипсетите на системния контролер в повечето по-нови системи. Наборът от чипове (82357 ISP или интегриран периферен контролер) и неговият интегрален набор от два DMA контролера, са програмирани точно както 8237. ISP /Програмируемият контролер на прекъсването/ също така предоставя двойка 8259A програмируеми контролери за прекъсване на системата.

DB7-DB₀ Крачетата на шината за данни са свързани към връзките на шината за данни на микропроцесора и се използват по време на програмирането на DMA контролера.

IOR Крачето IOR е двупосочен извод, използван по време на програмиране и по време на цикъл на DMA запис.

IOW Крачето IOW е двупосочен извод, използван по време на програмиране и по време на цикъл на DMA четене.

EOP Краят на процеса е двупосочен сигнал, който се използва като вход за прекратяване на DMA процес или като изход за сигнал за края на DMA трансфера. Този вход често се използва за прекъсване на DMA трансфер в края на DMA цикъл.

A₃-A₀ Тези адресни крачета избират вътрешен регистър по време на програмирането и също така предоставят част от адреса за DMA трансфер по време на DMA действие.

A₇-A₄ Тези адресни крачета са изходи, които осигуряват част от адреса за DMA трансфер по време на DMA.

HRQ Заявката за задържане е изход, който се свързва с входа HOLD на микропроцесора, за да поиска DMA трансфер.

DACK₃-DACK₀ Изходи за потвърждение на DMA канал. Те потвърждават заявка за DMA канал. Тези изходи могат да се програмират като активни- с високо или активни- с ниско ниво. DACK изходите често се използват за избор на DMA-управлявано I/O устройство по време на DMA трансфера.

AEN Сигналът за разрешаване на адреса активира ключа на DMA адреса, свързан към DB7-DB₀ крачета на 8237. Той също така се използва за деактивиране на всякакви буфери в системата, свързана към микропроцесора.

ADSTB Адресният строб функционира като ALE, с изключение на това, че се използва от DMA контролера за заключване на адресни битове A15-A₀ по време на DMA трансфера.

MEMR Четенето на паметта е изход, който кара паметта да чете данни по време на цикъл на четене на DMA.

MEMW Записът в паметта е изход, който кара паметта да записва данни по време на цикъл на запис на DMA.

Вътрешни регистри

CAR Текущият адресен регистър се използва за съхраняване на 16-битовия адрес на паметта, използван за DMA трансфера. Всеки канал има свой собствен регистър на текущите адреси за тази цел. Когато байт данни се прехвърля по време на DMA операция, CAR се увеличава или намалява, в зависимост от това как е програмиран.

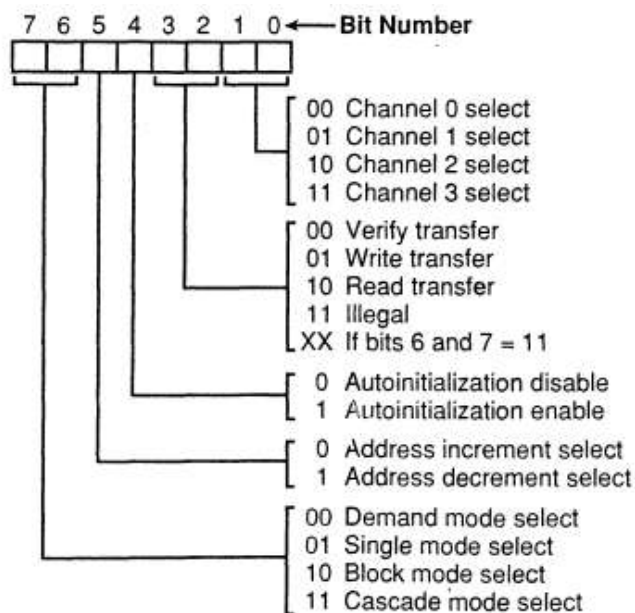
CWCR Текущият регистър на броя на думите програмира канал за броя байтове (до 64K), прехвърлени по време на DMA действие. Числото, заредено в този регистър, е с един по-малко от броя на прехвърлените байтове. Например, ако 10 се зареди в CWCR, тогава 11 байта се прехвърлят по време на действието DMA.

BA и BWC Регистрите на основния адрес (BA) и броя на основните думи (BWC) се използват, когато е избрана автоматична инициализация за канал. В режима на автоматична инициализация тези регистри се използват за презареждане както на CAR, така и на CWCR след завършване на действието DMA. Това позволява един и същ брой и адрес да се използват за прехвърляне на данни от една и съща област на паметта.

CR Командният регистър програмира работата на 8237 DMA контролера.

MR Режимният регистър програмира режима на работа за канал. Имайте предвид, че всеки канал има свой собствен регистър на режима (вижте Фигура 12-5), избран от позиции на битове 1 и 0. Останалите битове от регистъра на режима избират операцията, автоматичното инициализиране, увеличение/намаляване и режим за канала. Операциите за проверка генерират DMA адресите, без да генерират DMA памет и I/O контролни сигнали.

FIGURE 12-5 8237A-5 mode register (Courtesy of Intel Corporation)



Режимите на работа включват режим на търсене, единичен режим, блоков режим и каскаден режим.

Режимът на търсене прехвърля данни, докато не бъде въведен външен EOP или докато DREQ входът стане неактивен.

Единичен режим освобождава HOLD след прехвърляне на всеки байт данни. Ако крачетот DREQ се държи активно, 8237 отново изисква DMA трансфер през линията DRQ към входа HOLD на микропроцесора.

Блоковият режим автоматично прехвърля броя на байтовете, посочени от регистъра за броене за канала. DREQ не трябва да се поддържа активен чрез прехвърляне в блоков режим.

Каскадният режим се използва, когато в системата присъстват повече от един 8237.

RR Регистърът на заявката се използва за искане на DMA трансфер чрез софтуер.

MRSR Регистърът за установяване/нулиране на маските задава или изчиства маската на канала *i*.

MSR Регистърът на маската изчиства или задава всички маски с една команда вместо отделни канали, както при MRSR.

SR Регистърът на състоянието показва състоянието на всеки DMA канал (ТС битовете показват дали каналът е достигнал своя краен брой трансфери (ако е прехвърлил всичките си байтове). Когато се достигне крайния брой на трансферите, DMA прехвърлянето се прекратява за повечето режими на работа. Битовете на заявката показват дали DREQ входът за даден канал е активен.

Резюме за DMA

1. Входът HOLD се използва за искане на DMA действие, а изходът HLDA сигнализира, че задържането е в сила. Когато логическа 1 се установи на входа HOLD, микропроцесорът (I) спира изпълнението на програмата, (2) поставя своя адрес, данни и контролна шина в техните високоимпедансни състояния и (3) сигнализира, че задържането е в сила чрез установяване на логическа 1 на HLDA щифта.

2. Операция DMA четене прехвърля данни от място в паметта към външно I/O устройство. Операция DMA запис прехвърля данни от I/O устройство в паметта. Предлага се също прехвърляне от памет към памет, което позволява прехвърлянето на данни между две места на паметта с помощта на DMA техники.

3. Контролерът за директен достъп до памет (DMA) 8237 е четириканално устройство, което може да бъде разширено, за да включва допълнителни DMA канали за трансфер.

Периферни интерфейси в РС

Много приложения за РС изискват познаване на шините, разположени в персоналния компютър. Понякога главните платки от персонални компютри се използват като основни системи в индустриални приложения. Тези системи често изискват персонализирани интерфейси, свързани към една от шините на основната платка. Тук ще представим шината ISA (Industrial Standard Architecture - архитектура на индустриалния стандарт), локалната шина VESA и шината PCI (Peripheral Component Interconnect - взаимно свързване на периферни компоненти). Показани са и някои прости интерфейси към всяка от тези шинни системи като насока за проектиране.

ISA шина

ISA съществува от самото начало на IBM-съвместимата персонална компютърна система (около 1982 г.). Всяка карта от първия РС може да работи във всеки от най-модерните компютри, базирани на Pentium Pro. Това става възможно благодарение на интерфейса на шината ISA, който съществува във всички тези машини, който все още е съвместим с ранните персонални компютри. През годините ISA шината еволюира от оригиналния 8-битов стандарт към 16-битовия стандарт, който се намира в повечето по-модерни днешни системи. Имаше дори 32-битова версия, наречена шина EISA (разширена ISA), но тя почти изчезна. Това, което остава днес в повечето персонални компютри, е слот (връзка) на основната платка, който може да приема или 8-битова ISA карта, или 16-битова ISA карта с електроника. 32-битовите печатни платки са по-често PCI или, в някои по-стари машини, базирани на 80486, VESA карти.

8-битов ISA интерфейс е показан на Фигура 14-1 и илюстрира 8-битов ISA конектор, който съществуваше дънната платка на всички персонални компютърни системи (може да има и 16-битов конектор). Конекторът на шината ISA съдържа цялата демултиплексирана адресна шина (A19--A0) за 8088 система с 1MB памет, 8-битовата шина за данни (D7-D0) и четирите контролни сигнала MEMR, MEMW, IOR, и IOW за управление на I/O и всякаква памет, която може да бъде поставена на печатна платка с електроника. Блок памет рядко се поставя към карта на ISA шина, която може да работи на честота 8 MHz..

Други сигнали, които могат да бъдат полезни за I/O интерфейс, са линията на заявка за прекъсване IRQ2-IRQ7. IRQ2 се пренасочва към IRQ9 в съвременните системи и е обозначен така на Фигура 14-1.

Контролните сигнали на DMA канали 0-3 също присъстват на конектора. Входовете за DMA заявка са обозначени с DRQ1-DRQ3, а изходите за потвърждение на DMA са обозначени DACK0-DACK3. Входният сигнал DRQ0 липсва, защото ранните персонални компютри са използвали изхода DACK0 като сигнал за опресняване на DRAM, която може да се намира на ISA картата. Днес на това краче е изведен тактов сигнал с период 15,2 μ s. Останалите крачета са захранване и Reset-HY.

FIGURE 14-1 The 8-bit ISA bus

Back of Computer

Pin #

1	GND	IO CHK
2	RESET	D7
3	+5V	D6
4	IRQ9	D5
5	-5V	D4
6	DRQ2	D3
7	-12V	D2
8	OWS	D1
9	+12V	D0
10	GND	IO RDY
11	MEMW	AEN
12	MEMR	A19
13	IOW	A18
14	IOR	A17
15	DACK3	A16
16	DRQ3	A15
17	DACK1	A14
18	DRQ1	A13
19	DACK0	A12
20	CLOCK	A11
21	IRQ7	A10
22	IRQ6	A9
23	IRQ5	A8
24	IRQ4	A7
25	IRQ3	A6
26	DACK2	A5
27	T/C	A4
28	ALE	A3
29	+5V	A2
30	OSC	A1
31	GND	A0

Solder Side Component Side

VESA локална шина

Много по-добро решение за 32-битов интерфейс е локалната шина VESA. Шината EISA работи само на честота 8 MHz, докато локалната шина VESA работи на честота 33 MHz. Приложенията, изискващи високоскоростен трансфер на данни, ще ползват локалната шина VESA.

Локалната шина VESA, която е обща за видео и дискови интерфейси към персоналния компютър в базирани на 80486 системи. VESA локалната шина също е разширение на ISA шината. Разликата е, че локалната шина VESA не променя 16-

битовите ISA конектори; Добавя се трети конектор (съединител VESA) зад 16-битовия ISA конектор. Връзките на тази шина са много подобни на карта за слот EISA. Локалната шина VESA съдържа също 32-битова шина за адрес и данни за свързване на паметта или I/O към микропроцесора.

Peripheral Component Interconnect(PCI) шина

Шината PCI (взаимно свързване на периферни компоненти), която е практически единствената шина, която се намира в най-новите системи Pentium Pro и почти всички системи Pentium. ISA шината все още съществува във всички тези нови системи, но само като интерфейс за по-стари 8-битови и 16-битови интерфейсни карти. С времето ISA шината може да изчезне.

PCI шината е заменила локалната шина VESA поради нейните plug-and-play характеристики и способността си да функционира с 64-битова шина за данни. PCI интерфейсът съдържа серия от регистри, разположени в малко устройство с памет на PCI интерфейса, които съдържат информация за платката. Информацията в тези регистри позволява на компютъра автоматично да конфигурира PCI картата. Тази функция, наречена plug-and-play, вероятно е основната причина PCI шината да е станала толкова популярна в най-новите системи. Фигура 14-9 показва системната структура за PCI шината в персонална компютърна система.

Микропроцесорната шина е отделна и независима от PCI шината. Микропроцесорът се свързва към PCI шината чрез интегрална схема, наречена PCI мост. Това означава, че практически всеки микропроцесор може да бъде свързан към PCI шината, стига в системата да има вграден PCI контролер /или PCI мост/. Системата Apple Macintosh също използва PCI шина. IBM произвежда система Power-PC, която съдържа PCI шина. В бъдеще всички компютърни системи могат да използват една и съща шина.

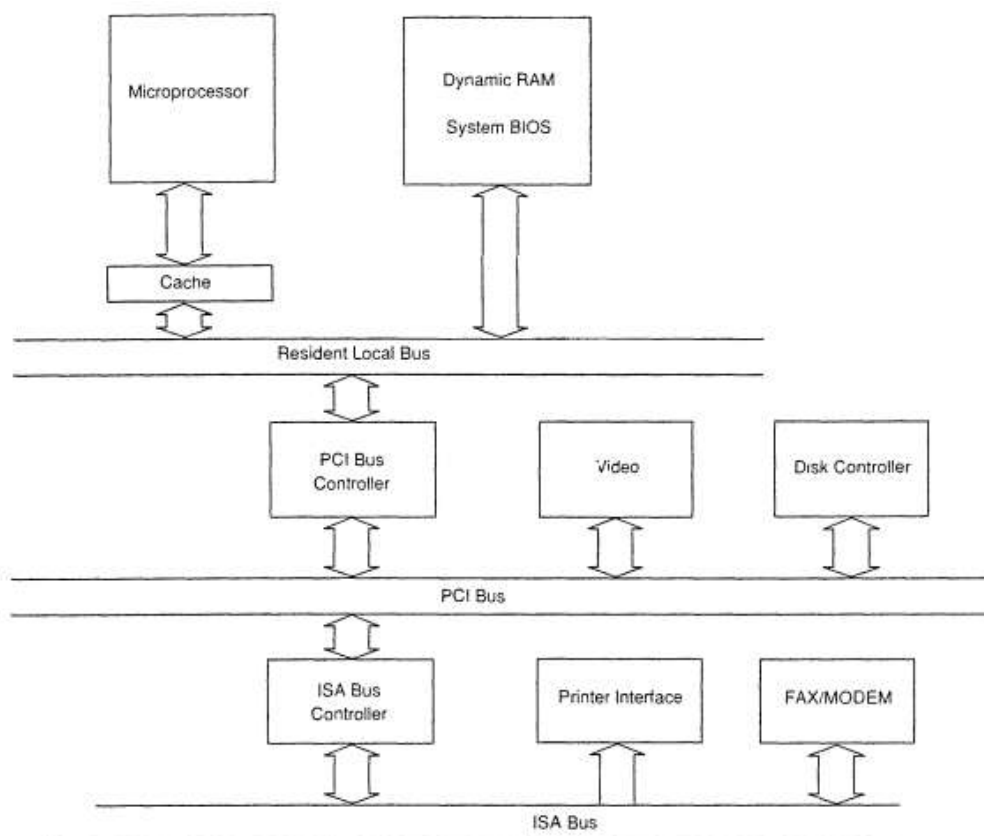


FIGURE 14-9 The system block diagram for the personal computer that contains a PCI bus

Както и други описвани шини, PCI шината съдържа всички управляващи сигнали за системата. За разлика от други шини, обаче, PCI шината функционира като 32-битова и 64-битова шина и адресира пълно 32-битово адресно пространство. PCI шината няма възможност за свързване към ISA конектор. Друга разлика е, че адресните и данните шини, които са мултиплексирани са изведени на съединител. Те са означени съответно като AD0-AD63. 32-битовите PCI карти имат 62 извода, докато при 64-битовите карти има 94 линии. 64-битовите карти могат да прехвърлят 64-битови адреси.

PCI шините се използват най-често за свързване на I/O компоненти към микропроцесора. Памет също може да бъде свързана и управлявана, но ще работи на 33 MHz, което е наполовина от скоростта на системната шина на Pentium или Pentium Pro системи.

Адреси и данни по PCI съединителите

PCI адресът се появява на линиите AD0-AD31 и е мултиплексиран с данните. В някои системи има 64-битова шина за данни, която използва AD32-AD63 само за пренос на данни, се използва и за разширяване на адреса до 64-бита.

Фигура 14-11 илюстрира времедиаграма за PCI шината, като показва начина, по който адресът се мултиплексира с данните, както и управляващите сигнали, използвани за мултиплексиране. По време на първия тактов период, адресът на паметта или I/O адрес се появява на AD линиите, а командата към PCI периферия се появява на C/BE линиите.

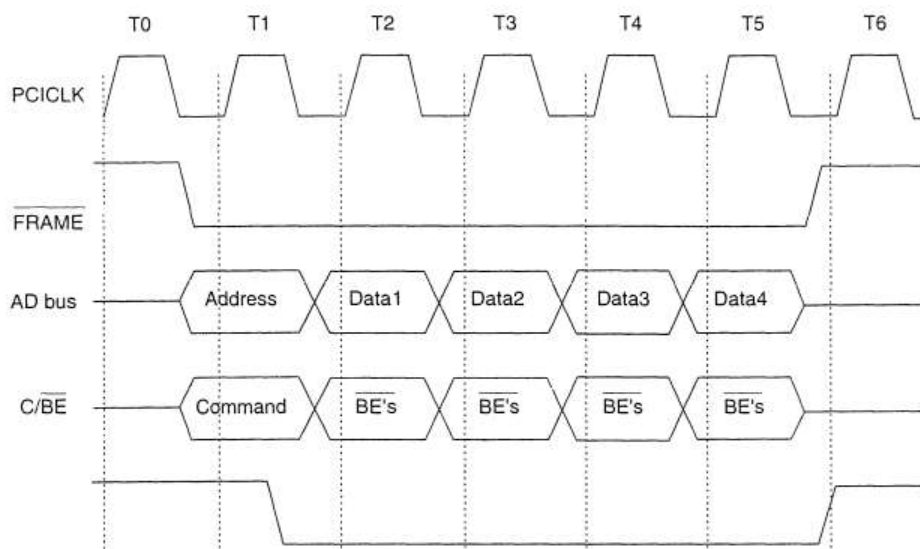


FIGURE 14-11 The basic burst mode timing for the PCI bus system. Note that this transfers either four 32-bit numbers (32-bit PCI) or four 64-bit numbers (64-bit PCI).

Таблица 14—4 илюстрира командите на шината, намиращи се на PCI шината.

TABLE 14-4 PCI bus commands

<i>C/BE3–C/BE0</i>	<i>Command</i>
0000	INTA sequence
0001	Special cycle
0010	I/O read cycle
0011	I/O write cycle
0100–0101	Reserved
0110	Memory read cycle
0111	Memory write cycle
1000–1001	Reserved
1010	Configuration read
1011	Configuration write
1100	Memory multiple access
1101	Dual addressing cycle
1110	Line memory access
1111	Memory write with invalidation

Команди за PCI шината

INTA последователност По време на потвърждаване на прекъсването, се адресира контролера на прекъсванията, който единствен генерира прекъсване и поставя вектор на прекъсване на шината за данни по време на операция четене на байт.

Специални цикли Тези цикли се ползват за трансфер на данни до всички PCI компоненти. През тези цикли най-десните 16 бита на данните съдържат 0000H, което индицира че процесорът прави HLT, 0001H ако се извършва операция Halt, или 0002H при специфични операции за кода и данните при 80x86 процесори.

Цикъл I/O четене Данните се четат от I/O устройство, използвайки I/O адреса, който се появява на линиите AD0-AD15. Многоциклово четене не се поддържа за I/O устройства.

Цикъл I/O запис Достъпът до I/O устройството става както при цикъла на I/O четене, но се прави запис на данни.

Цикъл на четене от паметта Четене на данни от памет, свързана на PCI шина.

Цикъл на запис в паметта Запис на данни в памет, свързана на PCI шина.

Четене на конфигурацията Информация за конфигурацията се чете от PCI устройство чрез цикъл на четене на конфигурацията.

Резюме

1. Системите от локални шини (ISA, EISA, VESA и PCI) позволяват I/O устройства и системите с памет да бъдат свързани към персоналния компютър.

2. ISA шината е 8- или 16-битова и поддържа трансфер с паметта или I/O при скорост от 8 MHz.

3. Шината EISA е разширена версия на шината ISA, която поддържа 8-, 16-, и 32-битов трансфер между персоналния компютър и паметта или I/O устройство със скорост от 8 MHz.

4. Локалната шина VESA (Video Electronics Standards Association) поддържа 32-битови трансфери между персоналния компютър и I/O или паметта при скорост от 33 MHz.

5. PCI шината поддържа 32- или 64-битови трансфери между персоналния компютър и паметта или I/O със скорост от 33 MHz. Тази шина позволява практически всеки микропроцесор да бъде свързан към PCI шината чрез използването на мостов интерфейс.

6. Plug-and-play интерфейс е този, който съдържа памет, която съхранява информация за конфигурацията на системата.

64-битова архитектура. Тенденции за развитие на микропроцесорите

В компютърната архитектура 64-битовите цели числа, адресите на паметта или други единици данни са тези, които имат 64-битова ширина. Също така, архитектурите на 64-битов централен процесор (CPU) и аритметично логическо устройство (ALU) са тези, които се основават на процесорни регистри, адресни шини или шини за данни с такъв размер. 64-битовите микрокомпютри са компютри, в които има 64-битови микропроцесори. От гледна точка на софтуера, 64-битовото изчисление означава използването на машинен код с 64-битови адреси на виртуална памет. Въпреки това, не всички 64-битови набори от инструкции поддържат пълни 64-битови адреси на виртуална памет; x86-64 и ARMv8, например, поддържат само 48 бита виртуален адрес, като останалите 16 бита от виртуалния адрес трябва да бъдат всички 0 или всички 1, а няколко 64-битови набора от инструкции поддържат по-малко от 64 бита физическа памет адрес.

Терминът 64-битов описва поколение компютри, в които 64-битовите процесори са стандартно вградени в системата. 64 бита е размер на думата, който дефинира определени класове компютърна архитектура, шини, памет и процесори и като разширение, софтуера, който работи върху тях. 64-битовите процесори се използват в суперкомпютрите от 70-те години на миналия век. Процесорните регистри обикновено са разделени на няколко групи: за цели числа, за числа с плаваща запетая, за единична инструкция, за една инструкция, но множество данни (SIMD), контролни и често специални регистри за адресна аритметика, които могат да имат различни приложения и имена като адресни, индексни или базови регистри. Въпреки това в съвременните проекти тези функции често се изпълняват от целочислени регистри с по-общо предназначение. В повечето процесори само цели числа или адресни регистри могат да се използват за адресиране на данни в паметта; другите видове регистри не могат. Следователно размерът на тези регистри обикновено ограничава количеството директно адресируема памет, дори ако има регистри, като регистри с плаваща запетая, които са по-широки. Повечето високопроизводителни 32-битови и 64-битови процесори (има някои изключения при по-стара или вградена ARM архитектура и 32-битова архитектура MIPS CPU) имат интегриран хардуер с плаваща запетая, който често, но не винаги, се базира на върху 64-битови даннови устройства. Например, въпреки че архитектурата x86/x87 има инструкции, които могат да зареждат и съхраняват 64-битови и 32-битови стойности с плаваща запетая в паметта, вътрешните данни с плаваща запетая и форматът на регистъра са с ширина 80 бита, докато общите регистри са широки 32 бита. 64-битовото семейство Alpha използва 64-битови данни и регистров формат с плаваща запетая и 64-битови целочислени регистри.

История

Много компютърни набори от инструкции са проектирани така, че един целочислен регистър може да съхранява адреса на паметта за всяко място във физическата или виртуалната памет на компютъра. Следователно общият брой адреси към паметта често се определя от ширината на тези регистри. IBM System/360 от 60-те години на миналия век е ранен 32-битов компютър; Той имаше 32-битови целочислени регистри, въпреки че използва само 24 бита на дума от нисък порядък за адреси, което води до адресно пространство от 16 MiB (16×10242 байта). 32-битовите супер миникомпютри, като DEC VAX, станаха често срещани през 70-те години на миналия век, а 32-битовите микропроцесори, като семейството Motorola 68000 и 32-битовите членове на семейството x86, започвайки с Intel 80386, се появиха в средата на 1980-те, което прави 32 бита удобен размер на регистъра. 32-битов адресен регистър

означаваше, че 2^{32} адреса, или 4GiB памет с произволен достъп (RAM), могат да бъдат адресирани. Когато тези архитектури бяха разработени, 4GiB памет беше толкова далеч над типичните количества (4MiB) в компютърните системи, че се смяташе, че това е достатъчно адресно пространство. Някои суперкомпютърни архитектури от 1970-те и 1980-те, като Cray-1, използваха регистри с ширина до 64 бита и поддържаха 64-битова целочислена аритметика, въпреки че не поддържаха 64-битово адресиране. Въпреки това, 32 бита остават норма до началото на 90-те години на миналия век, когато непрекъснатото намаляване на цената на паметта доведе до инсталации с количества RAM, приближаващи 4 GiB, а използването на пространства за виртуална памет, надвишаващи тавана от 4GiB, стана желателно за работа с определени видове проблеми. В отговор на това MIPS и DEC разработиха 64-битови микропроцесорни архитектури, първоначално за работни станции и сървърни машини от висок клас.

До средата на 90-те HAL Computer Systems, Sun Microsystems, IBM, Silicon Graphics и Hewlett Packard разработиха 64-битови архитектури за своите работни станции и сървърни системи. Забележително изключение от тази тенденция бяха мейнфреймите от IBM, които след това използваха 32-битови данни и 31-битови размери на адреса; мейнфреймите на IBM не включват 64-битови процесори до 2000 г. През 90-те години на миналия век няколко евтини 64-битови микропроцесора бяха използвани в потребителската електроника и вградените приложения. По-специално, Nintendo 64 и PlayStation 2 имаха 64-битови микропроцесори преди въвеждането им в персоналните компютри. Принтери от висок клас, мрежово оборудване и индустриални компютри също използват 64-битови микропроцесори, като Quantum Effect Devices R5000. 64-битовите изчисления започнаха да се ползват при персоналния компютър от 2003 г. нататък, когато някои модели Macintosh на Apple преминаха към процесори PowerPC 970, а Advanced Micro Devices (AMD) пусна първия си 64-битов x86-64 процесор.

Граници на процесорите

По принцип 64-битов микропроцесор може да адресира 16 EiB ($16 \times 1024^6 = 2^{64} = 18\,446\,744\,073\,709\,551\,616$ байта или около 18,4 ексабайта) памет.

Не всички набори от инструкции и не всички процесори, изпълняващи тези набори от инструкции, поддържат пълно 64-битово виртуално или физическо адресно пространство. Архитектурата x86-64 (от 2016 г.) позволява 48 бита за виртуална памет и, за всеки даден процесор, до 52 бита за физическа памет. Тези ограничения позволяват размери на паметта съответно от 256 TiB (256×1024^4 байта) и 4PiB (4×1024^5 байта). Понастоящем компютърът не може да съдържа 4 пебибайта памет (поради физическия размер на чиповете с памет), но AMD предвиждаше големи сървъри, споделени клъстери памет и други видове използване на физическо адресно пространство, които биха могли да се доближат до това в обозримо бъдеще. По този начин 52-битовият физически адрес осигурява достатъчно място за разширяване, като същевременно не поема разходите за внедряване на пълни 64-битови физически адреси. По същия начин, 48-битовото виртуално адресно пространство е проектирано така, че да осигури 2^{16} пъти 32-битовия лимит от 4 GiB (4×1024 байта), което позволява място за по-късно разширяване и не изисква излишни разходи за преобразуване на пълни 64-битови адреси. Архитектурата на системата за виртуална памет ARM AArch64 позволява 48 бита за виртуална памет и, за всеки даден процесор, от 32 до 48 бита за физическа памет DEC Alpha 21264 поддържа конфигурируеми от потребителя 43 или 48 бита адресно пространство на виртуална памет (8 TiB или 256 TiB) и 44 бита адресно пространство на физическа памет (16 TiB).

32-битова срещу 64-битова архитектура

Промяната от 32-битова към 64-битова архитектура е фундаментална промяна, тъй като повечето операционни системи трябва да бъдат значително модифицирани, за да се възползват от новата архитектура, тъй като този софтуер трябва да управлява действителния хардуер за адресиране на паметта. Друг софтуер също трябва да бъде пренесен, за да се използват новите възможности. По-старият 32-битов софтуер може да се поддържа или поради това, че 64-битовият набор от инструкции е надградено множество на 32-битовия набор от инструкции, така че процесорите, които поддържат 64-битовия набор от инструкции, могат също да изпълняват код за 32-битовия набор от инструкции или чрез софтуерна емуляция, или чрез реалното внедряване на 32-битово процесорно ядро в рамките на 64-битовия процесор, както при някои процесори Itanium от Intel, които включват процесорно ядро IA-32 за изпълнение на 32-битови x86 приложения. Операционните системи за тези 64-битови архитектури обикновено поддържат както 32-битови, така и 64-битови приложения.

На 64-битов хардуер с архитектура x86-64 (AMD64), повечето 32-битови операционни системи и приложения могат да работят без проблеми със съвместимостта. Докато по-голямото адресно пространство на 64-битовите архитектури прави работата с големи набори от данни в приложения като цифрово видео, научни изчисления и големи бази данни по-лесна, имаше значителен дебат дали те или техните 32-битови режими на съвместимост ще бъдат по-бързи от 32-битови системи на сравнима цена за други задачи. Компилирана Java програма може да работи на 32- или 64-битова виртуална машина на Java без никакви модификации. Дължините и прецизността на всички вградени типове, като char, short, int, long, float и double, и типовете, които могат да се използват като индекси на масиви, се определят от стандарта и не зависят от основния архитектура. Java програмите, които се изпълняват на 64-битова виртуална машина на Java, имат достъп до по-голямо адресно пространство.

Скоростта не е единственият фактор, който трябва да се вземе предвид при сравняването на 32-битови и 64-битови процесори. Приложения за многозадачност, стрес тестване и клъстериране – за високопроизводителни изчисления (HPC) – може да са по-подходящи за 64-битова архитектура, когато са употребени по подходящ начин. Поради тази причина 64-битовите клъстери са широко използвани в големи организации, като IBM, HP и Microsoft.

Предимства и недостатъци

Често срещано погрешно схващане е, че 64-битовите архитектури не са по-добри от 32-битовите, освен ако компютърът няма повече от 4 GiB памет с произволен достъп. Някои операционни системи и определени хардуерни конфигурации ограничават пространството на физическата памет до 3 GiB на IA-32 системи, поради голяма част от 3–4 GiB региона, запазен за хардуерно адресиране. 64-битовите архитектури могат да адресират много повече от 4 GiB. Въпреки това, процесорите IA-32 от Pentium Pro нататък позволяват 36-битово адресно пространство на физическа памет, използвайки Physical Address Extension (PAE), което дава 64 GiB физически адресен диапазон, от които до 62 GiB могат да се използват от основната памет. Операционни системи, които поддържат PAE, може да не са ограничени до 4 GiB физическа памет, дори на процесори IA-32. Въпреки това, драйверите и друг софтуер за режим на ядрото, още повече по-старите версии, може да са несъвместими с PAE. Това е причина 32-битовите версии на Microsoft Windows да бъдат ограничени до 4 GiB физическа RAM. Някои операционни системи запазват части от адресното пространство на процеса за използване на ОС, като ефективно намаляват общото адресно пространство, налично за

памет за потребителски програми. Например, 32-битовият Windows запазва 1 или 2 GiB (в зависимост от настройките) от общото адресно пространство за ядрото, което оставя само 3 или 2 GiB съответно от адресното пространство, достъпно за потребителски режим. Тази граница е много по-висока при 64-битови операционни системи. Файловете, управляващи картата на паметта, стават все по-трудни за разполагане в 32-битови архитектури, тъй като файловете от над 4 GiB стават все по-често срещани. Някои 64-битови програми, като кодери, декодери и софтуер за криптиране, могат да се възползват много от 64-битовите регистри, докато производителността на други програми, като например ориентираният към 3D графика, остава незасегната при превключване от 32-битов към 64-битова среда. Някои 64-битови архитектури, като x86-64 и AArch64, поддържат регистри с по-общо предназначение от техните 32-битови аналози. Това води до значително увеличение на скоростта за къси цикли, тъй като процесорът не трябва да извлича данни от кеша или основната памет, ако данните могат да се поберат в наличните регистри.

Достъпност на софтуера

64-битовите системи, базирани на x86, понякога нямат еквиваленти на софтуер, който е написан за 32-битови архитектури. Най-сериозният проблем в Microsoft Windows са несъвместими драйвери на устройства за остарял хардуер. Повечето 32-битови приложения могат да работят на 64-битова операционна система в режим на съвместимост, наричан още режим на емуляция, например Microsoft WoW64 Technology за IA-64 и AMD64.

64-битовата Windows Native Mode драйверна среда работи върху 64-битов NTDLL.DLL, който не може да извика 32-битов Win32 подсистемен код (често за устройства, чиято действителна хардуерна функция се емулира в софтуер за потребителски режим, като Winprinters). Тъй като 64-битовите драйвери за повечето устройства не бяха налични до началото на 2007 г. (Vista x64), използването на 64-битова версия на Windows се считаше за предизвикателство. Оттогава обаче тенденцията се премести към 64-битови изчисления, тъй като цените на паметта паднаха и използването на повече от 4 GiB RAM се увеличи. Повечето производители започнаха да предоставят както 32-битови, така и 64-битови драйвери за нови устройства, така че липсата на 64-битови драйвери престана да бъде проблем.

64-битови драйвери не бяха предоставени за много по-стари устройства, които следователно не можеха да се използват в 64-битови системи. Съвместимостта на драйверите беше по-малко проблем с драйверите с отворен код, тъй като 32-битовите можеха да бъдат модифицирани за 64-битова употреба.

Поддръжката за хардуер, създаден преди началото на 2007 г., беше проблематична за платформите с отворен код поради относително малкия брой потребители. 64-битовите версии на Windows не могат да изпълняват 16-битов софтуер. Въпреки това повечето 32-битови приложения ще работят добре с 16-битов софтуер.

64-битовите потребители са принудени да инсталират виртуална машина с 16- или 32-битова операционна система, за да изпълняват 16-битови приложения.

Резюме

64-битов процесор работи най-добре с 64-битов софтуер.

64-битов процесор може да има обратна съвместимост, което му позволява да изпълнява 32-битов приложен софтуер за 32-битовата версия на своя набор от инструкции и може също да поддържа работа с 32-битови операционни системи за 32-битовата версия на своя набор от инструкции.

32-битов процесор е несъвместим с 64-битов софтуер.